Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Recitation

Week 2

Prof. Lee Wee Sun

Summary

Do asymptotic analysis to ignore machine/ implementation dependent constants

- Growth of $T(n)$ as n approaches ∞.

Usually do **worst case analysis**.

Sometimes do **average case analysis**.

- Requires knowledge of input distribution

Write **recurrence** to describe running time of algorithms.

Use a **recursion tree** to solve recurrence.

Correctness

For iterative algorithms

- Invent a loop invariant. Show that it
  - Is true at initialization
  - Is correctly maintained
  - Implies correctness at termination
- Argue algorithm terminates, if necessary.

Correctness

For recursive algorithms

- Do mathematical induction, usually on size of problem
  - Show base case(s) true
  - Assuming cases smaller than *n* are correct, show that problem size *n* solved correctly

Admin

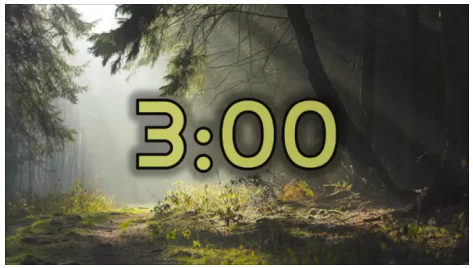
Reminder to complete the Pre-requisite Test by Sunday midnight.

MCQ

Question 1

A constant $n_0 \geq 1$ exists such that for any $n \geq n_0$, there is an array of n elements such that insertion sort runs faster than merge sort on that input.

- True
- False



Question 1

Answer: True

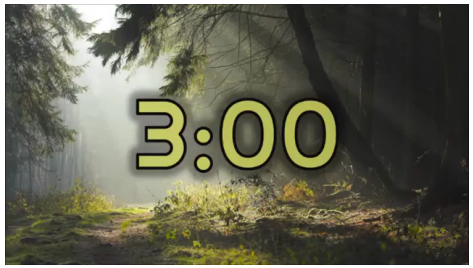
Best cases runtime of insertion sort is $O(n)$ when the array is *already sorted*.

Merge sort runs in time $\Theta(n \lg n)$ regardless of input.

http://visualgo.net/

Question 2

Use a recursion tree to give an asymptotically tight upper bound to the recurrence $T(n) = T(n-a) + T(a) + cn$ where $a \geq 1$ and $c > 0$, and $T(a)$ is a constant.

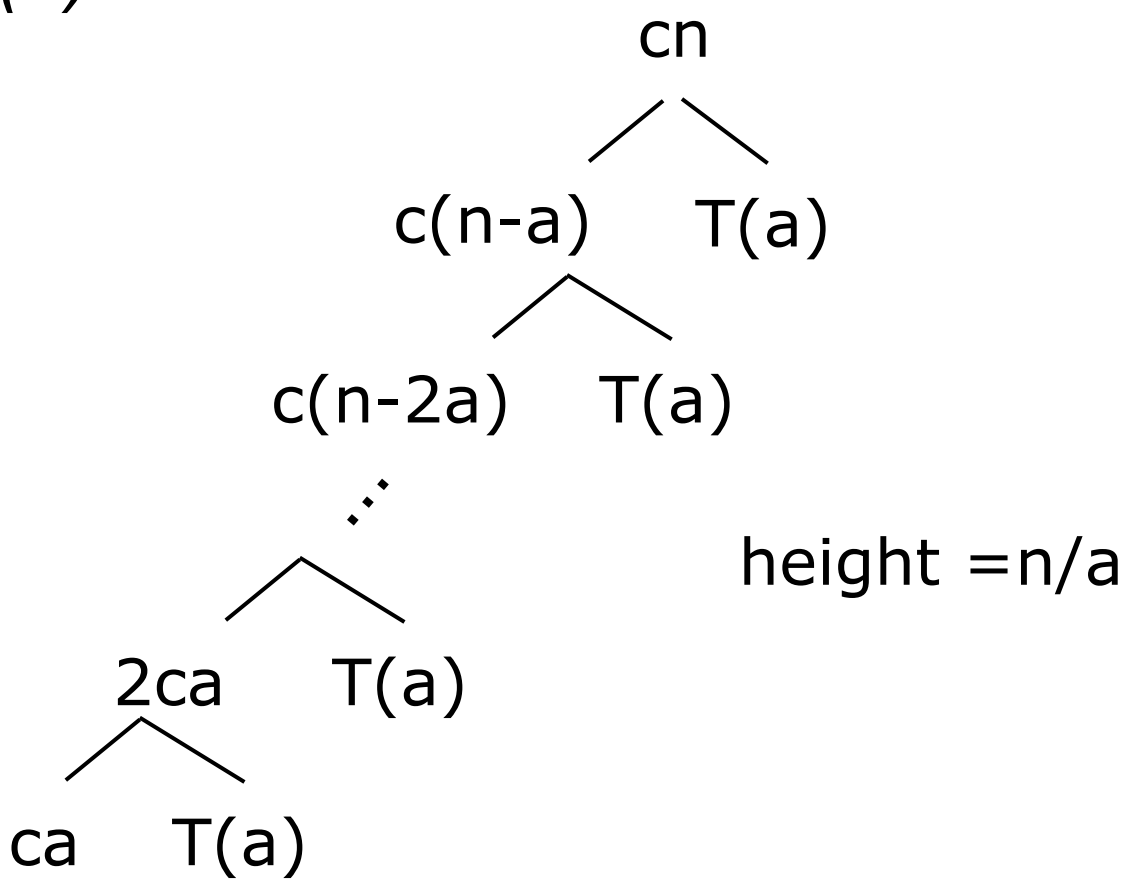


- $T(n) \leq C\frac{n}{a}\log n$ for some constant C
- $T(n) \leq Cn/a$ for some constant C
- $T(n) \leq Cn^2/a$ for some constant C
- $T(n) \leq Ca\log n$ for some constant C

Question 2

$T(n) = T(n\text{-}a) + T(a) + cn$

Answer: $T(n) \leq Cn^2/a$



$T(n) = T(n\text{-}a) + T(a) + cn$

Recursion tree height is n/a

At depth k, computation:

$T(a) + c(n\text{-}ka)$.

Summed over height, we get

$$T(a)\frac{n}{a} + ac + 2ac + 3ac + \cdots + \frac{n}{a}ac$$

$$= T(a)\frac{n}{a} + \frac{ac}{2}\frac{n}{a}\left(\frac{n}{a} + 1\right) \leq C\left(\frac{n^2}{a}\right)$$

Arithmetic Series

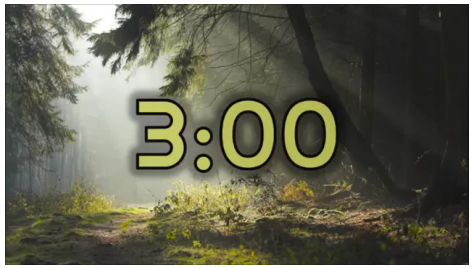
$$\sum_{k=1}^{n} k = 1 + 2 + 3 + \cdots + n$$

$$= \frac{1}{2}n(n+1) = \Theta(n^2)$$

Question 3

Use a recursion tree to determine an asymptotically tight upper bound for the recurrence $T(n) = T(n/2) + n^2$

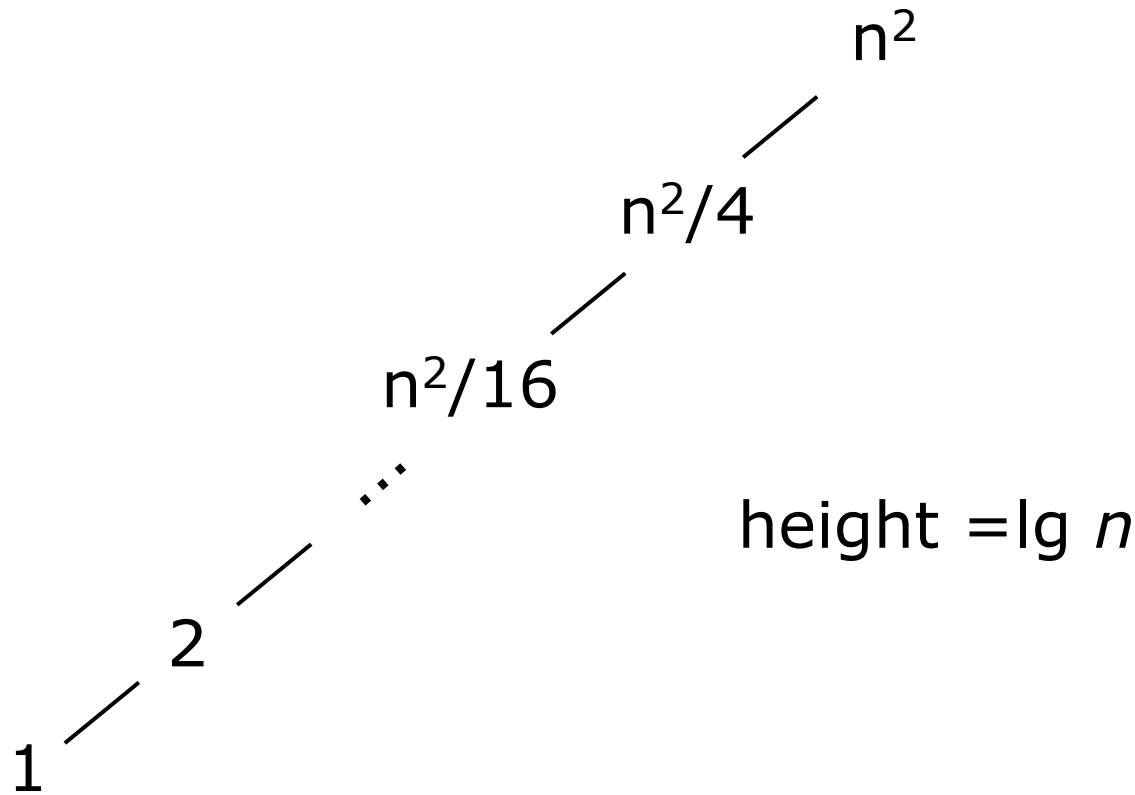
- $O(n^3)$
- $O(n)$
- $O(\log n)$
- $O(n^2)$



Question 3

$T(n) = T(n/2) + n^2$

Answer: $O(n^2)$



$T(n) = T(n/2) + n^2$

Recursion tree height is *lg n*
At depth *k*, total computation is $n^2/4^k$.
Summed over height, we get

$$n^2\left(1+\frac{1}{4}+\frac{1}{16}+\ldots\right)=O(n^2)$$

Geometric Series

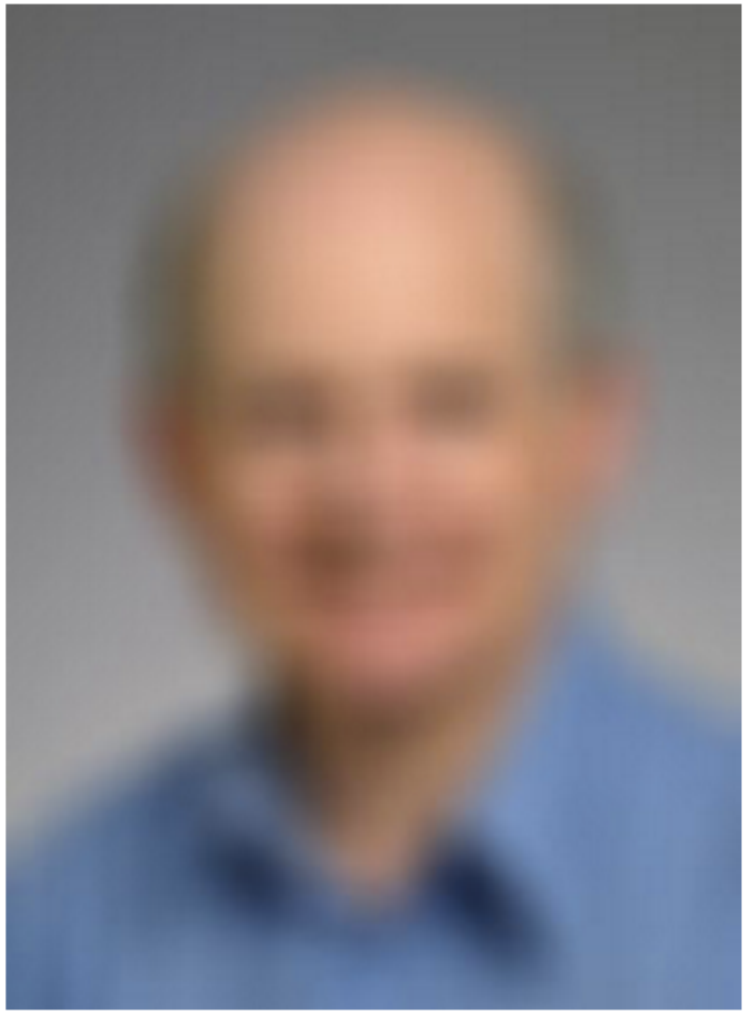
$$\sum_{k=1}^{n} x^k = 1 + x + x^2 + \cdots + x^n$$

$$= \frac{x^{n+1}-1}{x-1}$$

$$\sum_{k=1}^{\infty} x^k = \frac{1}{1-x} \text{ when } |x| < 1$$

Question 4

Who is the Master of Algorithms pictured below?



- Tom Cormen
- Charles Leiserson
- Ron Rivest
- Cliff Stein

Question 4

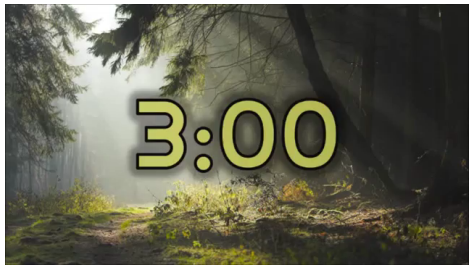
Answer: Charles E Leiserson

MIT Professor
Author of
Introduction to Algorithms
Well known for Cilk multithreaded language, fat-tree interconnection network used in supercomputers, cache oblivious algorithms, …

Question 5

You are given k sorted arrays, with n elements each, and you want to suitably combine them into a single sorted array (with kn elements).
One strategy to do this is to use a 2-way merge to merge the first two sorted arrays, then merge the result with the third sorted array, then merge with the fourth, and so on. What is the time complexity of this algorithm, in terms of k and n?

- $\Theta(k^2n^2)$
- $\Theta(k^2n)$
- $\Theta(kn^2)$
- $\Theta(kn)$



Question 5

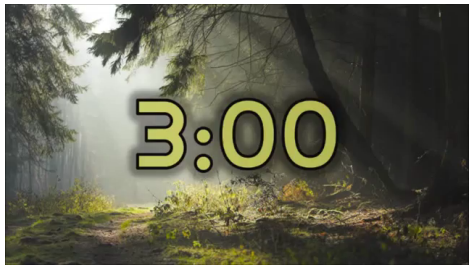
Answer: $O(k^2n)$

Total time to merge m elements is $\Theta(m)$.
Merging the first two lists takes time proportional to $2n$.
Merge another list takes time $3n$.
Then $4n$, $5n$, all the way to kn.
Total time is $2n+3n+ \ldots + kn = \Theta(k^2n)$

Question 6

The pseudo-code for selection sort is given below:

```
SELECTION-SORT(A)
n = A.length
for j = 1 to n-1
    smallest = j
    for i = j+1 to n
        if A[i] < A[smallest]
            smallest = i
    exchange A[j] with A[smallest]
```



What is a suitable loop invariant for the outer loop?

- The array A is sorted.
- The array A[1..j-1] is sorted.
- The array A[1..j-1] contains the j-1 smallest elements of the array A[1..n]
- The array A[1..j-1] is sorted and contains the j-1 smallest elements of the array A[1..n].

Question 6

Answer: The array $A[1..j\text{-}1]$ is sorted and contains the j-1 smallest elements of the array $A[1..n]$

Termination: The invariant needs to imply a sorted array when the loop terminates.

- At termination, $j == n$, so by the invariant, $A[1..n\text{-}1]$ is sorted and does not have elements larger than $A[n]$, implying that the whole array is sorted.

Initialization: $A[1..j\text{-}1]$ is empty, so invariant true.

Maintenance: Need invariant for inner loop stating that $A[smallest]$ is the smallest element in $A[j .. i\text{-}1]$.

When inner loop terminate, $i == n + 1$, so $A[smallest]$ is the smallest element in $A[j .. n]$.

If outer invariant is true before loop, it will be true after swapping on last line and incrementing j.